LoSeR: Long, Seasonal, and Residual Trend Forecasting for Univariate Stock Trading

Jefrey Bergl
Vibhas Nair
Rishabhdev Potti
Aditya Veerathu

Abstract

Time Series Forecasting (TSF) is an important application of autoregressive models that spans several key fields, such as energy, urban development, and financial markets. We focus on the stock price prediction aspect of these models and aim to build an application on top of an efficient TSF model. In TSF, it is important to capture long-range dependencies in data, to which end we tested multiple architectures and styles of forecasting models to select the lightest and most effective model for stock price forecasting. We propose the LoSeR model, which combines Long and Seasonal trends with a Residual component to perform long-horizon autoregression. On top of this model, we run an aggressive trading strategy based on multi-horizon inputs to simulate the potential profit/loss made by our system. We find that the LoSeR model using both the LSTM and SRU units are able to effectively inform a trading strategy that trades profitably on the S&P 500 index.

1 Introduction

Autoregressive models have historically been used to predict the future, beginning in the early 20th century. In 1970, Box and Jenkins proposed the ARIMA method [8], which relies on moving averages of a stationary time series to predict values. This method was quickly adopted for use in financial contexts, where fluctuating prices do not oscillate around a fixed average. These traditional models were later improved by neural network models, such as the Recurrent Neural Network (RNN) [14] and the Long Short-Term Memory (LSTM) unit [7]. The LSTM uses different types of gates to select which data is important to update its parameters. A further improvement to these recurrent models was the Statistical Recurrent Unit (SRU) [12], which uses fewer parameters to achieve comparable performance to LSTMs by utilizing the linear combination of averaged summary statistics. Leveraging these more recent neural network based units, we introduce the Long, Seasonal, and Residual (LoSeR) model, which combines linear regression components which model the long term and seasonal trends of the data with a recurrent model trained on the residual error from these two components. The LoSeR model is then used as the base for a trading strategy. To come in this paper is an analysis of previous work which inspired our approach, an explanation of the LoSeR models and trading strategy, and finally our testing results.

2 Related Work

2.1 Feature Selection

The LoSeR model trains based on raw OHLCV (Open, High, Low, Close, Volume) data for stock price forecasting, rather than transforming the feature space. This is inspired by the work in the Stock Price Prediction Using Triple Barrier Labeling paper [9]. Previous work referenced in this

paper makes use of engineered features to predict stocks, which is shown to be unnecessary by the authors. They apply a simple LSTM model to raw OHLCV data and achieve similar performance to models trained on engineered features. This approach inspired us to focus on using RNN-based frameworks as well as raw OHLCV data instead of engineered data. This approach simplifies our data preprocessing pipeline and allows the LoSeR model to be trained on readily available data directly without creating features.

2.2 Model

Our initial foray into investigating which model may be most effective was greatly assisted by a survey paper by Qiu et al. [13]. This paper offers a structured survey of deep learning approaches for multivariate time series forecasting (MTSF), with a unique focus on modeling correlations among channels (i.e., variables). It proposes a three-level taxonomy for channel strategies: channel independence (CI), channel dependence (CD), and channel partiality (CP). IThe survey also compares strengths, weaknesses, and implementation costs of each strategy, then outlines future research directions for foundation models and multimodal forecasting. This paper gave us an excellent understanding of the possibilities for model selection and techniques. Another inspiration for our model organization was the Prophet paper by Taylor et al. [15]. This paper proposes a decomposable time series model that consists of three components, a growth term, a seasonality term, and a holidays term. This model is focused towards business application, and seeks to forecast at scale. The decomposable forecast model influenced our decision to factor in the long trend and seasonal trend terms.

We initially selected the LSTM [7] unit as the recurrent unit for our model due to its lower parameter scale and accessibility. We were then introduced to the SRU architecture by Oliva et al. [12]. This architecture is an un-gated recurrent neural architecture that substitutes complex gates in LSTMs/GRUs with moving averages of learned summary statistics. SRUs retain long-term sequence dependencies by maintaining statistics at multiple scales, offering many viewpoints of recent and distant past data through simple linear operations. Empirical results show that SRUs often outperform LSTMs and GRUs, especially on tasks with profound long-term dependencies. SRUs capability to learn and maintain multi-scale dependencies may enhance temporal pattern recognition in stock data, especially for trading strategies sensitive to historical context. We test our model by training the residual term using both SRU and LSTM unit to determine if one will provide more accurate forecasting.

2.3 Trading Strategy

Our strategy synthesizes three established frameworks: multi-horizon forecasting, transaction cost aware execution, and differential weighting. Regarding the former, Zhang and Zohren[17] demonstrated that encoder decoder models that leverage multiple forecast horizons outperform single-horizon approaches, specifically at longer time scales. This result is particularly valuable to our project, as predicting multiple minute horizons from minute OHLCV data is considered longer term relative to high frequency trading strategies. Additionally, Wen et al.[16] showed that multi-horizon quartile forecasts improve decision-making under uncertainty by capturing different temporal dynamics simultaneously. This motivated using four LSTM horizon outputs rather than just one.

The differential weighting (horizon bias) derives from Garleanu and Pederson's[6] finding that predictors with faster mean reversion should dominate portfolio construction. Their "aim portfolio" framework explicitly weights signals by persistence. Specifically, since one minute predictions decay fastest, they receive the highest weight for immediate execution decisions. Mukherjee et al.[11]) also validated performance-based weighting for intraday trading, confirming that recency and relevance improve signal quality.

To simulate aspects of real market conditions, our strategy also accounts for transaction cost by implementing a threshold-based execution mechanism. Luu and Song[10] proved that optimal trading policies under fixed transaction costs require threshold-based decision schemes, where trades are only executed when profit exceeds friction. Specifically, we assume that the cost of a transaction does

not exceed 0.3% of its order's value. We chose this threshold value based on works such as Aked and Masturzo[1] and Frazzini et al.[5], which show that transaction costs are generally in the range of 0.3% to 0.75%.

3 Methods

3.1 Prediction model

We decompose time series forecasting into three parts: a long term trend for the data that spans over decades, a seasonal trend which represents weeks, months, or individual years, and a residual to capture day-to-day fluctuations. The equations for our model are shown below:

$$Y(t) = L(t) + S(t) + R(t)$$

$$L(t) = \beta_1 t + \beta_0$$

$$S(t) = \sum_{i=1}^{n} \alpha_i sin(nt) + \gamma_i cos(nt)$$

L(t) is the long trend component, represented using a linear term to model the general trend of the graph. S(t) is the seasonal component, which is decomposed into series of sine and cosine curves. This is essentially a truncated Fourier series. Finally the R(t) term is the residual term modeled by the selected recurrent neural unit that takes input from the time series features.

To train the model specifically, we convert the linear time space into a feature space that includes the exponential term for L(t) and the sinusoidal terms for S(t). Then, we perform a linear regression in that feature space on the altered feature space. To train R(t), we calculate the residual using R(t) = Y(t) - (L(t) + S(t)). This residual term is passed to the recurrent unit network followed by two dense layers.

3.2 Trading Strategy

3.2.1 Multi-Horizon Signal Construction

In this section we mathematically define our trading strategy. Let P_t denote the closing price at time t, and let $\hat{P}_{t+h}$ represent the model's predicted price at horizon $h \in \{1, 10, 30, 60\}$ minutes ahead, where predictions are made at the current time $t = 0$. We define the expected return for each horizon as:

$$r_h = \frac{\hat{P}_h - P_0}{P_0}, \quad h \in \{1, 10, 30, 60\}. \tag{1}$$

We then construct a composite signal S as a weighted linear combination of the predicted returns:

$$S = w_1 r_1 + w_{10} r_{10} + w_{30} r_{30} + w_{60} r_{60}, \tag{2}$$

where the weights must satisfy $\sum_h w_h = 1$ and will initially be set to $w_1 = 0.5$, $w_{10} = 0.3$, $w_{30} = 0.15$, and $w_{60} = 0.05$.

The weighting scheme reflects the empirical considerations that shorter-horizon forecasts exhibit provide more immediate actionable signals[6], and longer-horizon forecasts inform directional consistency and filter short-term noise. The decreasing weight structure balances immediacy against trend confirmation, ensuring that the strategy is not overly reactive to high-frequency fluctuations.

3.2.2 Transaction-Cost-Aware Thresholding

To prevent excessive trading in possible real world transaction costs, we implement a dynamic execution threshold τ that scales with forecast uncertainty. Trading occurs only when the composite

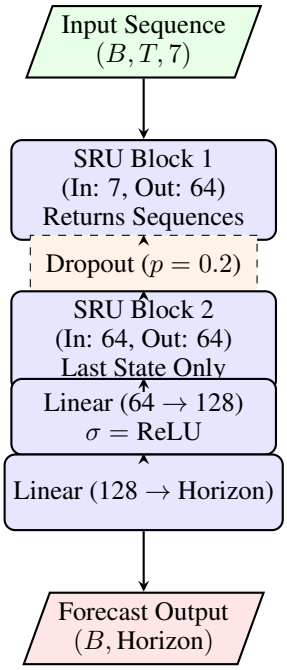

Figure 1: Architecture of the SRU Model. The model processes a 7-feature time series through two stacked SRU layers, extracting the final hidden state to predict the forecast horizon via a multi-layer perceptron.

signal exceeds this threshold in magnitude, ensuring that the expected edge justifies transaction costs. The threshold was originally defined as:

$$\tau = \max\left(0.003, \frac{2 \cdot \text{RMSE}}{P_0}\right), \tag{3}$$

where RMSE is the root mean squared error of the model's 1-minute-ahead predictions on the validation set, and P_0 is the current price. The lower bound of 0.003 (0.3%) represents a conservative estimate of transaction costs for large-cap equities[5][1]. The uncertainty-scaled component $2 \cdot \text{RMSE}/P_0$ ensures that the threshold expands when the model exhibits higher prediction variance, imposing stricter execution criteria during periods of uncertainty. After several tests, we determined that the following more aggressive threshold rules are more optimal for our test datasets:

$$\tau_{enter} = \max\left(0.0005, \frac{2 \cdot \text{RMSE}}{P_0}\right), \tag{4}$$

$$\tau_{exit} = \max\left(0.0002, \frac{2 \cdot \text{RMSE}}{P_0}\right), \tag{5}$$

$$\tau_{flip} = \max\left(0.0008, \frac{2 \cdot \text{RMSE}}{P_0}\right), \tag{6}$$

3.2.3 Position Logic and Execution Rules

At each timestep, we define a state $\pi_t \in \{\text{LONG}, \text{SHORT}, \text{HOLD}\}$ based on the composite signal S and threshold τ:

$$\pi_t = \begin{cases} \text{LONG}, & \text{if } S > \tau, \\ \text{SHORT}, & \text{if } S < -\tau, \\ \text{HOLD}, & \text{if } |S| \leq \tau. \end{cases} \tag{7}$$

The HOLD state represents either no position or continuation of the previous position when the signal remains inside the neutral zone $[-\tau, \tau]$. This prevents rapid position flipping due to small signal perturbations under the threshold boundary.

Position exits and reversals follow three rules:

1. **Reversal:** If the current position is LONG and $S < -\tau_{flip}$, exit the long position and enter a short position (and vice versa).

2. **Flattening:** If an active position exists and $|S| \leq \tau_{exit}$, close the position and return to HOLD.

3. **End-of-day liquidation:** Close all positions 5 minutes before market close.

3.3 Experiments

In order to evaluate which architecture would provide optimal results with our trading strategy, we prepared four models: Raw LSTM, Raw SRU, LoSeR(LSTM), and LoSeR(SRU). As the names imply, the latter two models are the LoSeR method with residuals being used to train an LSTM and an SRU setup respectively. We use the S&P 500 ETF 1-minute OLHCV Dataset from Kaggle [2] to train our models, selecting the last 100k rows with a 25k test set size. Each of the models is trained and then used to make predictions across the 25k datapoints, which we run our trading strategy on. From this testing we determine which of our four models perform with the highest profit/loss. The best two models are then selected for further testing on stock data from other markets. For this testing we use the IBOVESPA Brazilian Stock Exchange dataset [3] as well as the Nifty 50 VIX dataset [4] from Kaggle.

4 Results

Table 1: Comparative Profit and Loss (PnL) analysis of baseline versus LoSeR-enhanced architectures. The SRU variant of the LoSeR framework achieves the highest profitability.

Model Name	PnL
Raw LSTM	-1.82
Raw SRU	0.00
LoSeR (LSTM)	11.69
LoSeR (SRU)	**13.16**

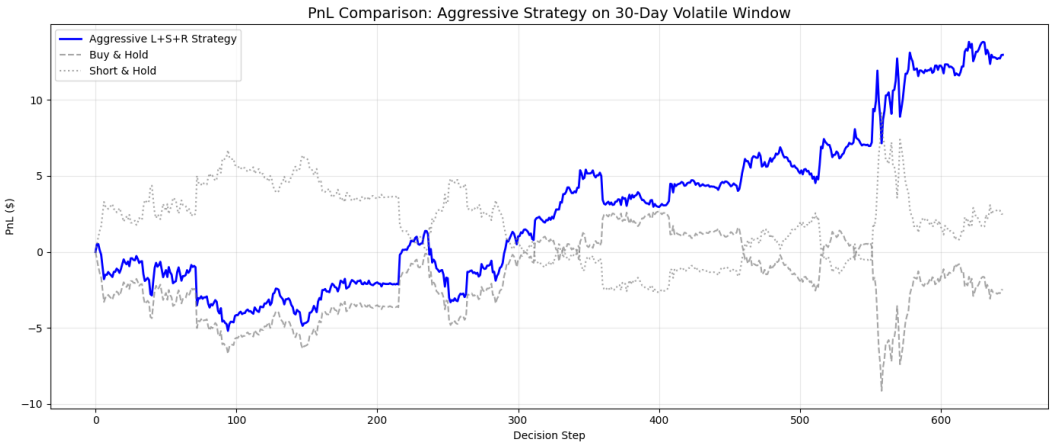

Figure 2: Graph of PnL for the LoSeR(SRU) Model on S&P 500 Data

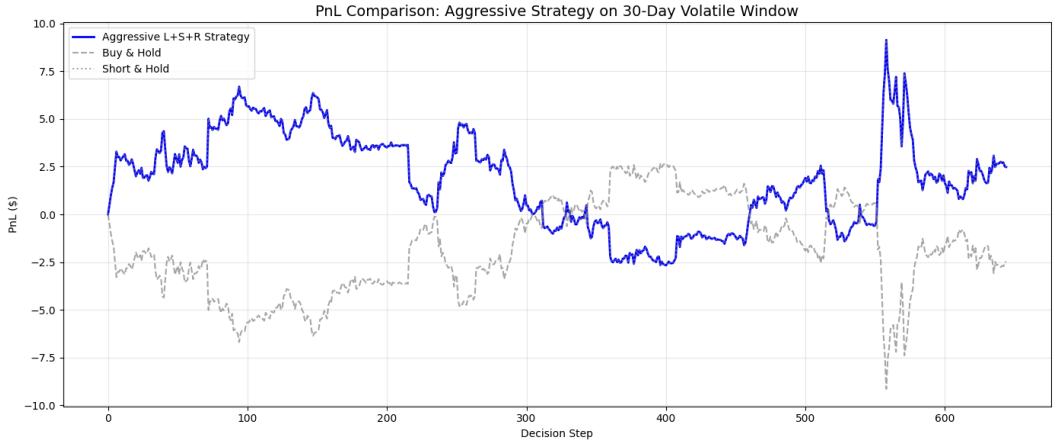

Figure 3: Graph of PnL for the Raw SRU Model on S&P 500 Data

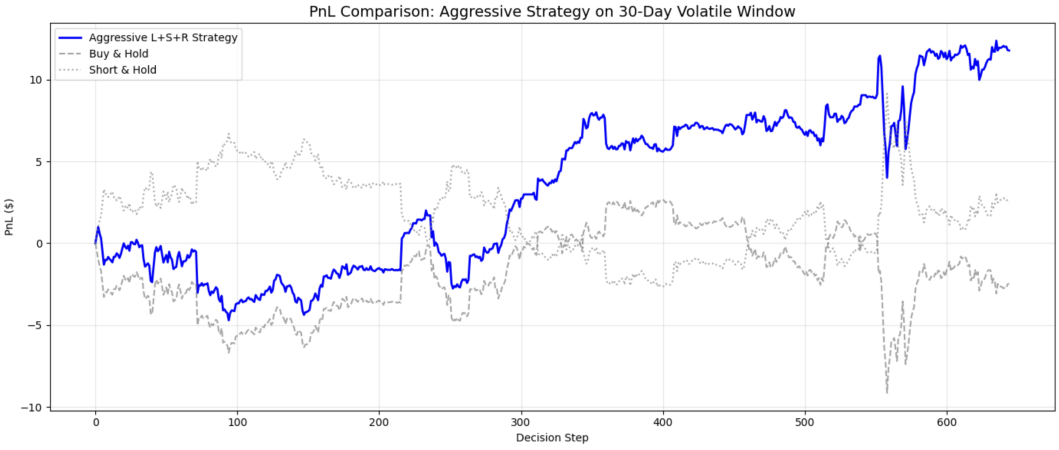

Figure 4: Graph of PnL for the LoSeR(LSTM) Model on S&P 500 Data

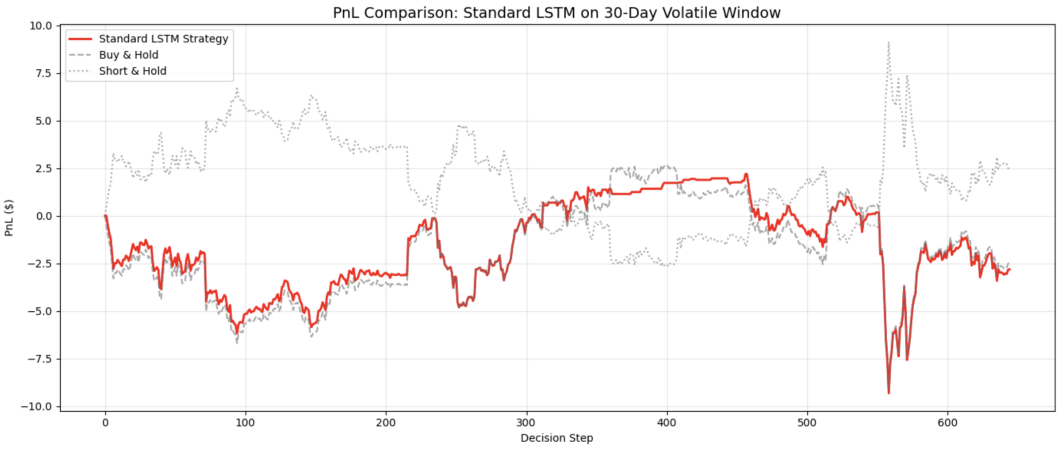

Figure 5: Graph of PnL for the Raw LSTM Model on S&P 500 Data

The table (1) above shows our results for testing on 25k rows from the S&P 500 dataset. The models used for this testing were: Raw SRU, Raw LSTM, LoSeR(LSTM), and LoSeR(SRU). From this testing we found that the LoSeR(LSTM) and LoSeR(SRU) outperform the other two models in terms of profit/loss. Visualized also are the PnL (profit/loss) charts for the four models (4) (2) (5) (3), where we can see that Raw LSTM is the real loser.

Table 2: Comparative Profit and Loss (PnL) analysis of LoSeR-enhanced architectures on unseen data.

Model Name	**Dataset**	**PnL (Local Currency)**
LoSeR (LSTM)	IBOVESPA	0.76
LoSeR (LSTM)	Nifty	-0.87

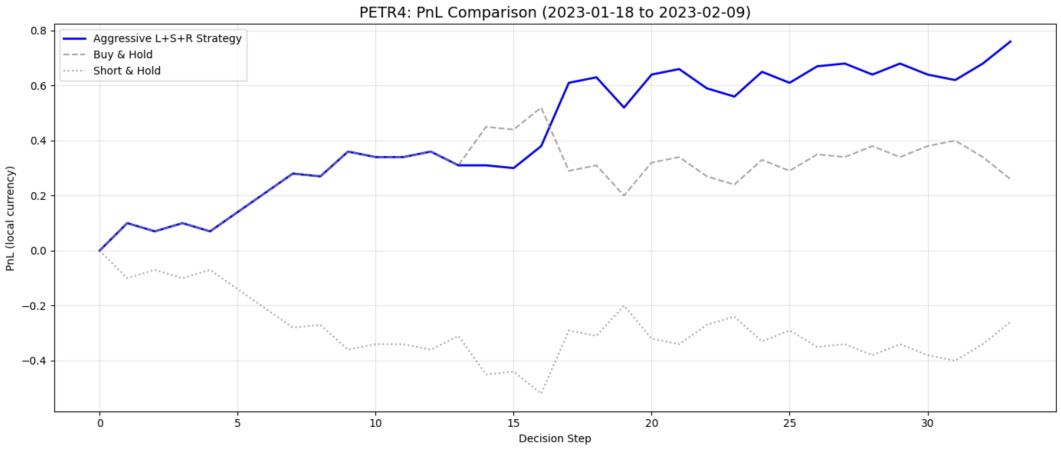

Figure 6: Graph of PnL for the LoSeR(LSTM) Model on IBOVESPA Dataset

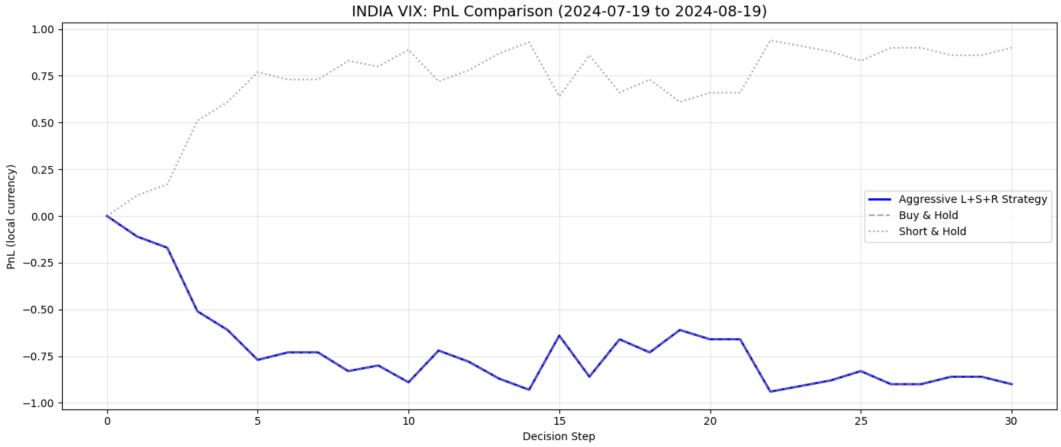

Figure 7: Graph of PnL for the LoSeR(LSTM) Model on Nifty Stock Data

The graphs above (6) (7) show the performance of our LoSeR(LSTM) and LoSeR(SRU) models on the IBOVESPA and Nifty stock data. We use 25k rows of data from each of the datasets on our two models to test. The PnL of this testing is also listed in table (2).

5 Limitations and Future Work

The LoSeR model, in combination with our trading strategy implementation, proved successful on the majority of our experiments, as represented by the figures above. Our strategy performed particularly well relative to simple hold and short strategies on periods of heightened price volatility. However, there are several possible improvements to our algorithm that would be interesting to explore. Firstly, we are curious to develop a rigorous parameter optimization approach for the trading model parameters. This would broadly involve creating an algorithm to update the thresholds based on a given history of market conditions or a desired risk tolerance. Both of these improvements require deeper knowledge of financial risk and complex optimization tasks, but present interesting problems to solve and allow the algorithm to provide more value.

It would also be interesting to examine exactly what market conditions led to the strategy's poor performance on the Nifty dataset (7). This analysis would provide insight into how to make the model more applicable to periods of varying conditions.
Another improvement that could be made is regarding the generalizability of the prediction models themselves. We find that the models trained on S&P 500 Data do not generalize well to the Brazilian or Nifty data. This could be rectified by training on a more diverse set of datasets while maintaining the current univariate structure or pivoting to a multivariate time series forecasting model which involves training on multiple stock indicator tickers simultaneously. This could tie into our trading strategy, allowing it to either focus on individual PnLs of the stocks or optimizing over the overall PnL of the portfolio.

6 Conclusion

We see that the LoSeR models outperform the raw LSTM and SRU models, which can be attributed to the inclusion of the long-term trend and seasonal components of training. Additionally, the trading strategy performs profitably using the predictions from our models. The strategy and predictions do not, however, generalize to unseen stock data, meaning that the LoSeR model cannot function as a general stock prediction model with the current amount of training data.

This paper contributes a novel decomposable time series prediction model that performs well on unseen test data that is similar to its training data. We also contribute a novel trading strategy based on several previous works. As mentioned previously, improvements can be made to the trading strategy by creating a dedicated optimization method for the parameters of the strategy. Overall, we find that lightweight models such as the LSTM and SRU are effective in financial market prediction applications despite their small number of parameters and relative shallowness in terms of layers. Additionally we see that adding traditional trend analysis contributes to the success of these models on limited training data.

References

[1] Michael Aked. Record low costs to trade! *Website Post*, 2016.

[2] Rockin Brock. Spy 1-minute. *Kaggle*.

[3] Antonio Brych. Brazilian stocks dataset: 5 years of 1 minute data. *Kaggle*.

[4] Deba. Nse - nifty 50 index minute data (2015 to 2025). *Kaggle*.

[5] Andrea Frazzini. Trading costs. *SSRN*, 2018.

[6] Nicola Gârleanu. Dynamic trading with predictable returns and transaction costs. *The Journal of Finance*, 2013.

[7] S Hochreiter. Long short-term memory. *Neural Computation, vol. 9*, 1997.

[8] Gwilym M Jenkins. Time series analysis; forecasting and control. *San Francisco : Holden-Day*, 1970.

[9] Sungwoo Kang. Stock price prediction using triple barrier labeling and raw ohlcv data: Evidence from korean markets. *arXiv preprint*, 2025.

[10] Phong Luu. A threshold type policy for trading a mean-reverting asset with fixed transaction costs. *Risks*, 2018.

[11] Aniruddha Mukherjee. Numin: Weighted-majority ensembles for intraday trading. *ACM International Conference on AI in Finance*, 2024.

[12] Junier Oliva. The statistical recurrent unit. *International Conference on Machine Learning*, 2017.

[13] Xiangfei Qiu. A comprehensive survey of deep learning for multivariate time series forecasting: A channel strategy perspective. *arXiv Preprint*, 2025.

[14] David Rumelhart. Learning internal representations by error propagation. *ICS*, 1985.

[15] Sean J. Taylor. Forecasting at scale. *The American Statistician*, 2018.

[16] Ruofeng Wen. A multi-horizon quantile recurrent forecaster. *arXiv preprint*, 2017.

[17] Zihao Zhang. Multi-horizon forecasting for limit order books: Novel deep learning approaches and hardware acceleration using intelligent processing units. *arXiv preprint*, 2021.